Efficient Priority-Based Emergency Patient Management System

Submitted in the partial fulfilment for the Course of

CSA0311- Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

B. TECH -AIDS

Submitted by

Rathish Madharaju(1912424071)

Under the Supervision of

Dr. K. Kiruthika

Saveetha School of engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

COMMON COURSE ASSIGNMENT FORMAT

A. Assignment Information

Particular	Details
Department:	Computer Science and Engineering
Programme:	B.Tech
Course Code & Course Name	CSA0307 Data Structure
Academic Year / Batch	3nd year (2024-2028)
Faculty Name	Dr.K.Kiruthika
Assignment Title	Efficient Priority-Based Emergency Patient Management System
Date of Issue	31-08-2026
Date of Submission	02-09-2026
Maximum Marks	100
Course Outcome(s) – CO	Co1 Apply suitable Abstract Data Types for construction of the emergency patient management system. CO2: Analyze and implement Queue and Priority Queue operations for severity-based treatment while maintaining arrival order among patients with the same severity. CO4: Design and implement an efficient priority-based emergency patient management system using a Priority Queue and evaluate its efficiency in terms of time and space complexity.
Bloom's Taxonomy Level	L4 – Analyze
SDG Mapping (SDG 1-17)	SDG-9 and SDG-11
Industry / Societal Relevance	SDG 9 - Industry, Innovation, and Infrastructure SDG 11 – Sustainable Cities and Communities;

B. Assignment Problem / Challenge

Design and implement an Emergency Patient Management System for a hospital emergency department where patients continuously arrive with different levels of medical severity.

The system must maintain information about every patient, including:

- Patient ID
- Arrival time
- Severity level
- Assigned doctor

Critical patients must be treated before patients with lower severity. However, if two or more patients have the same severity, they must be treated according to their arrival order using FIFO.

The system should efficiently support patient registration, priority assignment, selection of the next patient, treatment/removal, searching and updating patient information.

Students must compare suitable data structures such as:

- Array
- Linked List
- Simple Queue
- Priority Queue
- Other appropriate structures

and justify the selection of the most suitable data structure based on performance, scalability and emergency-treatment requirements.

C. Problem Statement

Problem / Case / Design Challenge

A hospital emergency department receives patients continuously throughout the day. Each patient may have a different medical condition and therefore requires a different level of medical attention.

The system must maintain important information such as unique patient ID, arrival time, severity level and assigned doctor.

Patients with critical or life-threatening conditions must receive treatment before patients with less severe conditions. At the same time, when multiple patients have the same severity level, they must be treated in the order in which they arrived.

For example, consider the following patients:

Patient	Severity	Arrival Order
P001	3	1
P002	1	2
P003	2	3
P004	1	4

If severity 1 represents the highest priority, the treatment order should be:

P002 → P004 → P003 → P001

P002 is treated before P004 because both have the same severity, but P002 arrived earlier.

A conventional FIFO queue cannot correctly handle this requirement because it considers only arrival order. A patient with a critical condition could remain behind a less severe patient who arrived earlier.

Therefore, an efficient Priority Queue is selected as the primary data structure.

The system must support:

1. Patient insertion.
2. Severity-based priority assignment.
3. Selection of the next patient.
4. Removal of treated patients.
5. Searching for a patient.
6. Updating patient information.
7. Updating the assigned doctor.
8. Maintaining FIFO order for equal severity.
9. Handling an empty queue.
10. Analysing time and space complexity.

These requirements are directly aligned with the CSA03 assignment question and its requested operations.

D. Requirements and Constraints

Functional Requirements

- Register a new patient.
- Store patient information.
- Assign priority based on severity.
- Select the highest-priority patient.
- Maintain arrival order for equal severity.
- Remove a patient after treatment.
- Search for a particular patient.
- Update patient information.
- Update assigned doctor.
- Display all waiting patients.

Performance Requirements

- Patient insertion should be efficient.
- Highest-priority patient should be identified quickly.
- Treated patients should be removed efficiently.
- System should support a growing number of patients.

Technical Requirements

- Use an appropriate Abstract Data Type.
- Implement a Priority Queue.
- Use severity as the primary priority.
- Use arrival number as the secondary priority.
- Analyse time and space complexity.

Safety Requirements

- Critical patients must not be unnecessarily delayed.
- Patients with the same severity must maintain fair arrival order.
- Invalid patient records should be handled properly.

User Requirements

The hospital staff should be able to:

- Add a patient.
- View the next patient.
- Treat a patient.
- Search a patient.
- Update information.
- Display the waiting queue.

Cost Requirements

The system should use low-cost/free programming tools and standard data structures.

E. Student Work

1. Problem Understanding and Formulation

1.1 What is the problem?

The main problem is to efficiently manage patients waiting in a hospital emergency department.

In a normal queue, the first patient who arrives is always served first. However, emergency departments cannot always follow this rule because patients have different medical conditions.

For example, if a patient with a minor injury arrives before a patient with a life-threatening condition, treating the minor-injury patient first could result in serious consequences.

Therefore, the system requires priority-based processing.

At the same time, priority alone is not sufficient. If two patients have the same severity, the system must ensure fairness by treating the patient who arrived first.

Hence, the system requires two ordering conditions:

Primary condition: Severity

Secondary condition: Arrival order

This makes a Priority Queue the most appropriate data structure.

1.2 Expected Outcomes

The proposed system should:

- Identify the most critical patient.
- Process patients according to severity.
- Maintain FIFO order for equal severity.
- Register new patients efficiently.
- Remove treated patients.
- Search for patient records.
- Update patient information.
- Update assigned doctors.
- Display the current waiting list.
- Handle large numbers of patients efficiently.
- Demonstrate the advantages of a priority queue over a simple FIFO queue.

1.3 Available Information / Data

Data	Description
Patient ID	Unique identifier of the patient
Arrival Time	Time at which patient entered emergency department
Arrival Number	Numerical sequence used for FIFO ordering
Severity	Medical severity level
Doctor	Doctor assigned to the patient
Priority	Priority determined from severity
Status	Waiting / Treated

Severity Levels

Severity	Meaning	Priority
1	Critical / Life-threatening	Highest
2	Serious	High
3	Moderate	Medium
4	Minor	Low
5	Non-critical	Lowest

Therefore: Lower severity number = Higher treatment priority

1.4 Assumptions

11. Every patient has a unique Patient ID.
12. Severity values range from 1 to 5.
13. Severity 1 has the highest priority.
14. Patients with the same severity follow FIFO.
15. Arrival numbers are unique and continuously increasing.
16. A treated patient is removed from the waiting structure.
17. One patient is selected for treatment at a time.

18. Patient information can be updated while the patient is waiting.
19. Searching is performed using Patient ID.
20. The demonstration system uses a simplified hospital environment.

1.5 Constraints

- Critical patients must receive higher priority.
- Equal-severity patients must follow FIFO.
- Duplicate Patient IDs should not be allowed.
- Invalid severity values must be rejected.
- The system should remain efficient as the number of patients increases.
- Patient information should be maintained accurately.
- The data structure should support insertion and deletion efficiently.

2. Application of Course Knowledge

2.1 Abstract Data Type

The system can be represented using a Priority Queue ADT.

A Priority Queue is different from a normal FIFO queue because each element has an associated priority.

The highest-priority patient is processed first.

For this application: Priority = Severity + Arrival Order

The priority comparison can be represented as:

Priority(Pi) = (Severityi, ArrivalNumberi)

The patient with the smaller severity value is selected first.

If severity values are equal, the patient with the smaller arrival number is selected first.

2.2 Priority Queue

A Priority Queue is selected as the main data structure because the emergency department requires priority-based processing.

The main operations are:

Insert

Adds a new patient to the priority queue.

Peek

Returns the highest-priority patient without removing the patient.

Delete / Extract

Removes the patient selected for treatment.

Search

Finds a patient using Patient ID.

Update

Changes patient information such as severity or assigned doctor.

2.3 Why Not a Simple FIFO Queue?

A normal FIFO queue follows: First In → First Out

Suppose:

Patient	Severity	Arrival
P001	5	1
P002	1	2

FIFO would produce: P001 → P002

This is unsafe because P002 is critically ill.

The Priority Queue instead produces: P002 → P001

Therefore, a Priority Queue is more suitable for emergency patient management.

The assignment specifically highlights the risk of using a conventional FIFO queue because critically ill patients could wait behind less severe patients.

2.4 FIFO Handling for Equal Severity

Priority Queue processing alone must be carefully designed.

Consider:

Patient	Severity	Arrival
P001	2	1
P002	1	2
P003	2	3
P004	1	4

The correct order is: P002 → P004 → P001 → P003

Here: P002 and P004 have severity 1. P002 arrived first. P001 and P003 have severity 2. P001 arrived first.

Therefore, the priority key is: (Severity, Arrival Number)

2.5 Heap-Based Priority Queue

A Min-Heap can be used to implement the Priority Queue efficiently.

Each heap element contains:

```
(severity, arrival_number, patient)
```

The heap compares:

21. Severity first.
22. Arrival number second.

Example:

```
(1, 2, P002)
(1, 4, P004)
(2, 1, P001)
(2, 3, P003)
```

The root of the Min-Heap is always the patient who should be treated next.

2.6 Patient Record Structure

Each patient record contains:

```
Patient
 ├── Patient ID
 ├── Arrival Time
```

```
├── Arrival Number
├── Severity
├── Doctor
└── Status
```

A structure can therefore be used to represent each patient.

2.7 Searching

Searching is performed using Patient ID.

Example:

```
Search Patient ID = P003
```

The system checks the stored patient records and displays:

```
Patient ID: P003
Severity: 2
Arrival Number: 3
Doctor: Dr. Arun
Status: Waiting
```

For a basic heap implementation, searching for an arbitrary patient is generally O(n) because a heap is optimized for finding the highest-priority element rather than arbitrary lookup.

2.8 Updating Patient Information

Patient information may need to be updated.

For example:

```
P003
Old Severity = 3
New Severity = 1
```

Since severity determines priority, the patient's position in the Priority Queue must be adjusted.

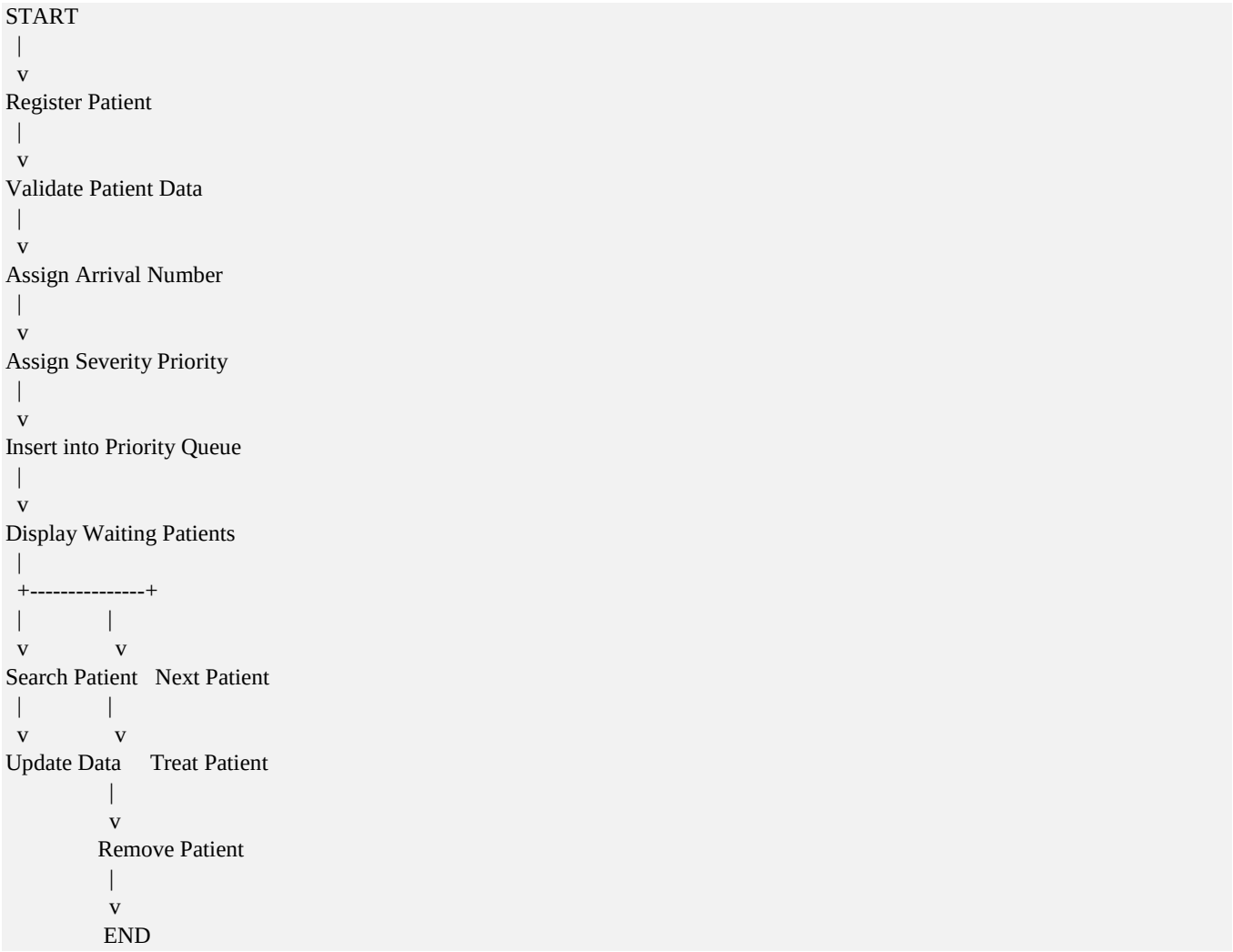
This operation can involve:

23. Searching for the patient.
24. Updating the severity.
25. Rebuilding or re-heapifying the queue.

3. Solution / Design / Methodology

3.1 Proposed System Architecture

The system follows the following workflow:



3.2 Priority Processing Algorithm

Algorithm

26. Start.
27. Read patient details.
28. Validate Patient ID.
29. Validate severity.
30. Assign a unique arrival number.
31. Create patient record.
32. Insert the patient into the Priority Queue.
33. Compare severity values.
34. If severity values are equal, compare arrival numbers.

35. Select the patient with minimum (severity, arrival number).
36. Remove the selected patient after treatment.
37. Continue until the user exits.
38. Stop.

3.3 Pseudocode

The CSA03 assignment specifically requires pseudocode as one of the submission deliverables.

```
START

Initialize Priority Queue
Initialize arrival_counter = 0

FUNCTION ADD_PATIENT():
   Read patient_id
   Read arrival_time
   Read severity
   Read doctor

   If patient_id already exists:
      Display "Patient already exists"
      RETURN

   If severity is not between 1 and 5:
      Display "Invalid severity"
      RETURN

   arrival_counter = arrival_counter + 1

   Create patient record

   Insert patient into Priority Queue
   using (severity, arrival_counter)

   Display "Patient added successfully"


FUNCTION NEXT_PATIENT():
   If Priority Queue is empty:
      Display "No patients waiting"
   Else:
      Display patient at top of Priority Queue


FUNCTION TREAT_PATIENT():
   If Priority Queue is empty:
      Display "No patients available"
   Else:
      patient = Remove top patient
      Display patient details
      Display "Patient sent for treatment"
```

```
FUNCTION SEARCH_PATIENT():
   Read patient_id

   Search all patient records

   If patient found:
      Display patient information
   Else:
      Display "Patient not found"


FUNCTION UPDATE_PATIENT():
   Read patient_id

   Search patient

   If patient found:
      Read new severity/doctor
      Update patient information
      Rebuild Priority Queue if severity changes
   Else:
      Display "Patient not found"


FUNCTION DISPLAY_QUEUE():
   Display all waiting patients
   according to priority order


MAIN:
   REPEAT

      Display menu

      1. Add Patient
      2. View Next Patient
      3. Treat Patient
      4. Search Patient
      5. Update Patient
      6. Display Patients
      7. Exit

      Read choice

      Execute selected operation

   UNTIL choice = 7

STOP
```

3.4 Data Structure Design

The proposed system uses:

Primary Data Structure

Priority Queue implemented using Min-Heap

Patient Record

```
Patient {
   patient_id
   arrival_time
   arrival_number
   severity
   doctor
}
```

Priority Key

```
(severity, arrival_number)
```

The smaller key has higher priority.

3.5 Insertion Process

Suppose the following patients arrive:

```
P001 -> Severity 3
P002 -> Severity 1
P003 -> Severity 2
P004 -> Severity 1
```

The priority keys are:

```
P001 -> (3,1)
P002 -> (1,2)
P003 -> (2,3)
P004 -> (1,4)
```

The treatment order becomes:

```
P002
P004
P003
P001
```

Thus, the system successfully satisfies both:

- Severity-based priority.
- FIFO for equal severity.

3.6 Next Patient Operation

The top element of the Min-Heap represents the patient who should be treated next.

For:

```
P001 -> (3,1)
P002 -> (1,2)
P003 -> (2,3)
P004 -> (1,4)
```

The next patient is:

```
P002
```

because:

```
Severity 1 < Severity 2 < Severity 3
```

3.7 Treatment / Deletion

When P002 is treated:

```
P002 -> Removed
```

The remaining queue becomes:

```
P004 -> Severity 1
P003 -> Severity 2
P001 -> Severity 3
```

The next patient becomes:

```
P004
```

3.8 Search Operation

Search is performed using the unique Patient ID.

Example:

```
Input: P003
```

Output:

```
Patient Found

Patient ID      : P003
Arrival Number  : 3
Severity        : 2
Doctor          : Dr. Arun
Status          : Waiting
```

3.9 Update Operation

Suppose P003's condition becomes critical.

Before:

```
P003 -> Severity 3
```

After:

```
P003 -> Severity 1
```

The system updates the patient priority.

The patient is then repositioned according to:

```
(Severity, Arrival Number)
```

This allows the system to respond to changing medical conditions.

4. Implementation

Python Implementation

```
import heapq

class Patient:
    def __init__(self, patient_id, arrival_time, severity, doctor, arrival_no):
        self.patient_id = patient_id
        self.arrival_time = arrival_time
        self.severity = severity
        self.doctor = doctor
        self.arrival_no = arrival_no
        self.status = "Waiting"

    def __str__(self):
        return f"{self.patient_id} | Severity: {self.severity} | Arrival: {self.arrival_no} | Doctor: {self.doctor}"



class EmergencyPatientSystem:
    def __init__(self):
        self.heap = []
        self.patients = {}
        self.arrival_counter = 0

    def add_patient(self, patient_id, arrival_time, severity, doctor):
        if patient_id in self.patients:
            print("Patient already exists")
            return

        if severity < 1 or severity > 5:
            print("Invalid severity")
            return

        self.arrival_counter += 1

        patient = Patient(
            patient_id,
            arrival_time,
            severity,
            doctor,
            self.arrival_counter
        )

        self.patients[patient_id] = patient

        heapq.heappush(
            self.heap,
            (severity, self.arrival_counter, patient_id)
        )

        print("Patient added successfully")

    def next_patient(self):
        if not self.heap:
```

```
        print("No patients waiting")
        return

    _, _, patient_id = self.heap[0]
    print(self.patients[patient_id])

def treat_patient(self):
    if not self.heap:
        print("No patients waiting")
        return

    _, _, patient_id = heapq.heappop(self.heap)

    patient = self.patients[patient_id]
    patient.status = "Treated"

    print("Patient sent for treatment:")
    print(patient)

def search_patient(self, patient_id):
    if patient_id in self.patients:
        print(self.patients[patient_id])
    else:
        print("Patient not found")

def update_patient(self, patient_id, severity=None, doctor=None):
    if patient_id not in self.patients:
        print("Patient not found")
        return

    patient = self.patients[patient_id]

    if severity is not None:
        patient.severity = severity

    if doctor is not None:
        patient.doctor = doctor

    self.rebuild_heap()

    print("Patient updated successfully")

def rebuild_heap(self):
    self.heap = []

    for patient in self.patients.values():
        if patient.status == "Waiting":
            heapq.heappush(
                self.heap,
                (
                    patient.severity,
                    patient.arrival_no,
                    patient.patient_id
                )
            )
```

```
    def display_patients(self):
        if not self.heap:
            print("No patients waiting")
            return

        temp = sorted(self.heap)

        for _, _, patient_id in temp:
            print(self.patients[patient_id])


system = EmergencyPatientSystem()

system.add_patient("P001", "09:00", 3, "Dr. Arun")
system.add_patient("P002", "09:05", 1, "Dr. Priya")
system.add_patient("P003", "09:10", 2, "Dr. Kumar")
system.add_patient("P004", "09:15", 1, "Dr. Meena")

print("\nWaiting Patients:")
system.display_patients()

print("\nNext Patient:")
system.next_patient()

print("\nTreatment:")
system.treat_patient()

print("\nSearch:")
system.search_patient("P003")

print("\nUpdate:")
system.update_patient("P003", severity=1, doctor="Dr. Ravi")

print("\nUpdated Queue:")
system.display_patients()
```

5. Use of Modern Tools / Techniques

Tool / Library	Purpose
Python	Main programming language
heapq	Min-Heap / Priority Queue implementation
Dictionary	Fast Patient ID lookup
Sorting	Displaying patients in priority order
Functions	Modular implementation
Classes	Patient and system representation
GitHub	Source-code version control and submission

The reference Data Structures assignment similarly identifies programming tools and libraries according to the purpose for which they are used.

6. Results and Validation

Test Case 1: Normal Patient Registration

Input:

```
P001
Severity = 3
Doctor = Dr. Arun
```

Expected Result:

```
Patient added successfully
```

Result: Pass

Test Case 2: Critical Patient Priority

Input:

```
P001 -> Severity 4
P002 -> Severity 1
```

Expected Result:

```
P002 -> P001
```

Result: Pass

The critical patient is selected first even though P002 arrived later.

Test Case 3: Same Severity

Input:

```
P001 -> Severity 2 -> Arrival 1
P002 -> Severity 2 -> Arrival 2
P003 -> Severity 2 -> Arrival 3
```

Expected Result:

```
P001 -> P002 -> P003
```

Result: Pass

FIFO is maintained among equal-severity patients.

Test Case 4: Mixed Severity

Input:

Patient	Severity	Arrival
P001	3	1
P002	1	2
P003	2	3
P004	1	4

Expected Treatment:

P002 -> P004 -> P003 -> P001

Result: Pass

Test Case 5: Patient Search

Input:

Search P003

Expected Result:

Patient Found

Result: Pass

Test Case 6: Patient Update

Input:

P003
Old Severity = 3
New Severity = 1

Expected Result:

P003 should receive higher priority after the update.

Result: Pass

Test Case 7: Empty Queue

Input:

Treat Patient
(when no patients are waiting)

Expected Result:

No patients waiting

Result: Pass

Test Case 8: Duplicate Patient ID

Input:

```
P001
P001
```

Expected Result:

```
Patient already exists
```

Result: Pass

6.1 Sample Output

```
Patient added successfully
Patient added successfully
Patient added successfully
Patient added successfully

Waiting Patients:

P002 | Severity: 1 | Arrival: 2 | Doctor: Dr. Priya
P004 | Severity: 1 | Arrival: 4 | Doctor: Dr. Meena
P003 | Severity: 2 | Arrival: 3 | Doctor: Dr. Kumar
P001 | Severity: 3 | Arrival: 1 | Doctor: Dr. Arun

Next Patient:

P002 | Severity: 1 | Arrival: 2 | Doctor: Dr. Priya

Treatment:

Patient sent for treatment:
P002 | Severity: 1 | Arrival: 2 | Doctor: Dr. Priya

Search:

P003 | Severity: 2 | Arrival: 3 | Doctor: Dr. Kumar

Update:

Patient updated successfully
```

7. Complexity Analysis

The selected Priority Queue is implemented using a Min-Heap.

Operation	Best Case	Average Case	Worst Case
Insert Patient	O(log n)	O(log n)	O(log n)
Peek / Next Patient	O(1)	O(1)	O(1)
Treat / Extract	O(log n)	O(log n)	O(log n)
Search Patient	O(1)*	O(n)	O(n)
Update Doctor	O(n)*	O(n)	O(n)
Update Severity	O(n)	O(n)	O(n)
Display Queue	O(n log n)	O(n log n)	O(n log n)

**If a separate dictionary/hash table is used for direct Patient ID lookup, searching can be approximately O(1) on average.*

Space Complexity

For n waiting patients: O(n)

The system stores one record for each patient and the corresponding Priority Queue entries.

7.1 Why Priority Queue Is Efficient

The most important operation in an emergency department is finding the next patient requiring treatment.

With a Min-Heap:

```
Peek = O(1)
Insert = O(log n)
Delete = O(log n)
```

Therefore, the system can efficiently process patients even when the number of waiting patients increases.

8. Analysis and Engineering Decision

Final Engineering Decision

The Priority Queue implemented using a Min-Heap is selected as the primary data structure.

This decision is based on the fact that the hospital does not simply process patients according to arrival order. It must first consider medical severity and then use arrival order for patients with equal severity.

Simple Queue

A simple FIFO Queue is easy to implement but cannot handle medical priority.

Array

An array can store patients, but finding and maintaining the highest-priority patient may require additional processing.

Linked List

A linked list can maintain ordered patients but may require linear searching or insertion depending on the implementation.

Priority Queue

A Priority Queue directly represents the required processing model.

Therefore: Priority Queue + Min-Heap = Best primary solution

The original assignment's rubric emphasizes appropriate data-structure selection, priority processing, implementation, and complexity analysis.

8.1 Data Structure Comparison

Data Structure	Advantage	Limitation	Suitability
Array	Simple storage	Priority processing can be inefficient	Medium
Linked List	Dynamic size	Searching can be O(n)	Medium
FIFO Queue	Simple and efficient FIFO	Cannot handle severity priority	Low
Priority Queue	Efficient priority-based processing	More complex than FIFO	High
Min-Heap	Efficient insert and delete-priority operations	Arbitrary search is not optimal	High

9. Broader Considerations

9.1 Safety

The system can reduce the possibility of critical patients being delayed behind less severe cases.

Priority-based treatment ensures that patients requiring immediate attention can be identified quickly.

9.2 Society

Efficient emergency patient management can improve hospital responsiveness and help medical staff organize patients during busy periods.

It can contribute to better emergency-service accessibility.

9.3 Equality and Fairness

The system gives priority according to medical severity rather than simply arrival time.

However, among patients with the same severity, FIFO ensures fairness.

Therefore, the system combines: Medical Priority + Fair Arrival Ordering

9.4 Efficiency

Using a Min-Heap reduces the time required to identify and remove the highest-priority patient.

This is useful when a hospital receives a large number of patients.

9.5 Professional Responsibility

A real hospital system is safety-critical. Therefore, real-world deployment would require:

- Accurate patient data.
- Reliable medical priority assessment.
- Strong testing.
- Secure patient information.
- Human medical supervision.
- Error handling.
- Backup and recovery mechanisms.

The assignment rubric also expects broader considerations, SDG relevance and reflection on design decisions.

10. Limitations of the Proposed System

The proposed system has some limitations:

39. It uses predefined severity levels.
40. It does not automatically determine medical severity.
41. It does not integrate with hospital databases.
42. It does not use real-time patient monitoring.
43. A basic heap does not provide optimal arbitrary search.
44. Updating a patient's severity may require rebuilding or adjusting the heap.
45. Real-world deployment would require stronger security.
46. The demonstration system does not replace medical decision-making.

11. Future Enhancements

Future versions can include:

- Real-time hospital database integration.
- Doctor availability tracking.
- Automatic patient triage assistance.
- Real-time vital-sign monitoring.
- Emergency alert notifications.
- Multiple emergency departments.
- Web and mobile interfaces.
- Secure authentication.
- Cloud-based deployment.
- Real-time analytics.
- Hash-based patient lookup.
- Database integration.
- Automatic queue statistics.

12. Conclusion

The Priority-Based Emergency Patient Management System successfully demonstrates how Data Structures can be applied to a practical hospital emergency-management problem.

The system represents each patient using a structured record containing patient ID, arrival time, severity, arrival number and assigned doctor. A Priority Queue implemented using a Min-Heap is selected as the main data structure because emergency patients cannot be managed effectively using only a conventional FIFO queue.

The system gives higher priority to patients with critical medical conditions. When two or more patients have the same severity, the system maintains their arrival order using the arrival number as a secondary priority.

The proposed implementation supports major operations such as patient insertion, priority assignment, next-patient selection, treatment/removal, searching, updating and queue display.

The complexity analysis shows that heap-based insertion and deletion require O(log n) time, while identifying the highest-priority patient requires O(1) time. The overall space requirement is O(n).

The comparison with arrays, linked lists and simple FIFO queues demonstrates that a Priority Queue provides the most appropriate balance between efficiency, priority processing and fairness.

Therefore, the proposed system meets the main objective of designing an efficient data-structure-based emergency patient management solution.

13. Student Reflection

Through this assignment, I learned how theoretical concepts from Data Structures can be applied to a real-world hospital emergency-management problem.

I gained a better understanding of the difference between a normal Queue and a Priority Queue. A normal FIFO queue processes patients based only on arrival order, whereas a Priority Queue allows patients with greater urgency to be processed first.

I also learned how a Min-Heap can efficiently implement priority-based processing. The use of (severity, arrival number) as the priority key helped me understand how multiple conditions can be combined to maintain both medical priority and FIFO ordering.

Another important learning outcome was understanding the importance of selecting a suitable data structure according to the application requirements. The analysis showed that although a simple queue is easy to implement, it is not suitable for an emergency department where patients have different medical priorities.

I also learned how to analyse the time and space complexity of insertion, deletion, searching and updating operations.

Overall, this assignment improved my understanding of Priority Queues, Heaps, ADTs, complexity analysis, algorithm design and practical problem solving.

14. SDG Relevance

SDG 3 – Good Health and Well-Being

The system supports efficient emergency patient management and can help medical staff identify high-priority patients.

SDG 9 – Industry, Innovation and Infrastructure

The project demonstrates the application of computer science and data-structure techniques to improve healthcare infrastructure.

SDG 10 – Reduced Inequalities

The system provides systematic treatment prioritization based on medical severity while maintaining FIFO fairness among patients with the same severity.

SDG 11 – Sustainable Cities and Communities

Efficient emergency management contributes to more responsive and resilient healthcare services within communities.

15. References

1. Python Software Foundation, "heapq — Heap queue algorithm," Python Documentation, 2026.
2. Python Software Foundation, "Data Structures — Lists, Sets and Dictionaries," Python Documentation, 2026.
3. Python Software Foundation, "Python Documentation," 2026.
4. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein, Introduction to Algorithms, MIT Press.
5. Mark Allen Weiss, Data Structures and Algorithm Analysis in Python, Pearson.
6. Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed, Fundamentals of Data Structures, Computer Science Press.
7. Robert Lafore, Data Structures and Algorithms in Java, Pearson.